

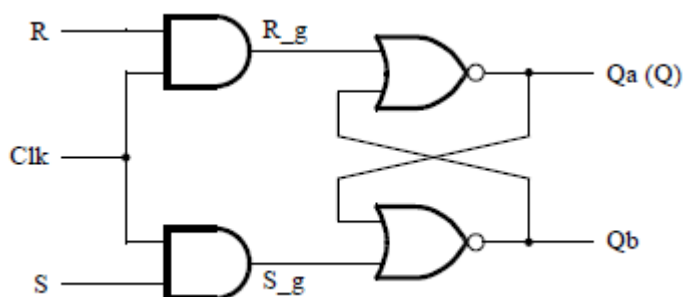
Lab. 5. Zatrzaski, przerzutniki, rejestry

Wymagana wiedza

- zatrzaski, przerzutniki, rejestry (rodzaje, synteza, działanie);
- parametry, parametry w deklaracji modułu, tworzenie instancji modułu;
- wielostronne operatory `?: :: {};` `{{}};`;
- Dyrektywa **keep**;
- Operacje sterowania czasem **posedge** i **negedge**;
- priorytet sygnałów sterujących przerzutnika / zatrzasku / rejestru.

Wykonanie

1. Utwórz dwa projekty zatrząskó RS z rys. 1.



Rys. 1. Układ zatrząsku RS utworzony z bramek logicznych

Jeden projekt **latch RS_gates** (rys. 2) używając prymitywów bramek języka **Verilog**, a drugi **latch RS_boolean** (rys. 3) za pomocą wyrażeń logicznych.

```
// bramkowy zatrząsk RS
module RS_gates (Clk, R, S, Q);
    input Clk, R, S;
    output Q;
    wire R_g, S_g, Qa, Qb    /* synthesis keep */;
    and (R_g, R, Clk);
    and (S_g, S, Clk);
    nor (Qa, R_g, Qb);
    nor (Qb, S_g, Qa);
    assign Q = Qa;
endmodule
```

Rys. 2. Opis zatrząsku RS, poprzez wykorzystanie instancji elementów logicznych

```
// bramkowy zatrząsk RS
module RS_boolean (Clk, R, S, Q);
    input Clk, R, S;
    output Q;
    wire R_g, S_g, Qa, Qb /* synthesis keep */;
    assign R_g = R & Clk;
    assign S_g = S & Clk;
```

```

assign Qa = ~(R_g | Qb);
assign Qb = ~(S_g | Qa);
assign Q = Qa;
endmodule

```

Rys. 3. Definicja zatrzasku RS przy użyciu wyrażeń logicznych

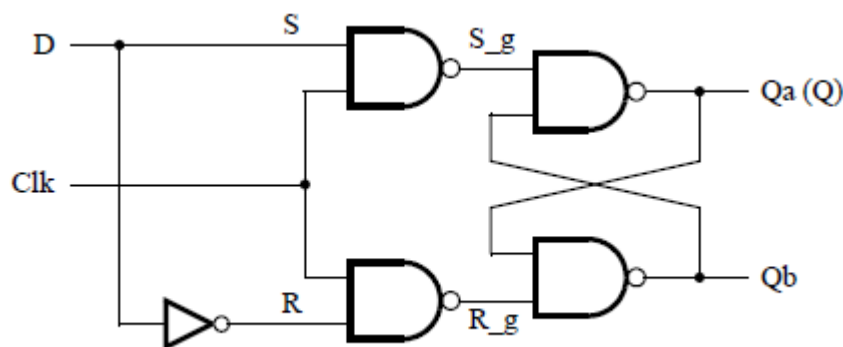
Wykorzystaj dyrektywę **keep** Aby monitorować wewnętrzne węzły projektu w symulacji. Porównaj wyniki syntezy (**RTL Viewer**, **Technology Map Viewer**) bez używania i z użyciem dyrektywy **keep**.

Wskazówka: Symulator nie daje komunikatu o błędzie z początkowymi wartościami CLK = 1 i R = 1. Sprawdź, co podaje symulator dla R = 1 i S = 1. Zaimplementuj projekt na płycie z następującymi pinami:

Wyprowadzenia	Piny
clk	SW[2]
R	SW[1]
S	SW[0]
Q	LEDR[0]

2. Utwórz projekt latch_D (rys. 4), używając dowolnego ze stylów opisu (używając prymitywów lub wyrażeń logicznych). Wykonaj wszystkie kroki z poprzedniego zadania dla tego projektu. Przydzielenie pinów:

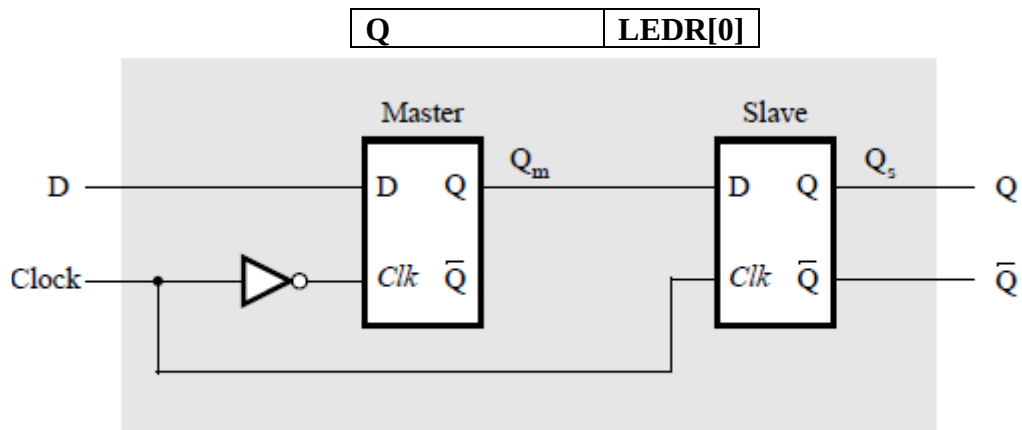
Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Q	LEDR[0]



Rys. 4. Schemat zatrzasku D

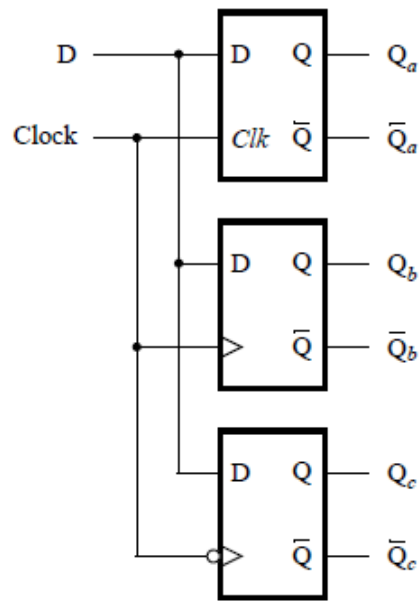
3. Utwórz projekt master_slave_D (rysunek 5) przerzutnika D z dwóch zatrzasków. Użyj dyrektywy **keep** do monitorowania wewnętrznych węzłów. Porównaj wyniki syntezy (**RTL Viewer**, **Technology Map Viewer**) bez użycia i korzystania z dyrektywy **keep**. Uruchom symulację. Przy jakich wartościach początkowych symulator wyświetla komunikat o błędzie? Zaimplementować projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]

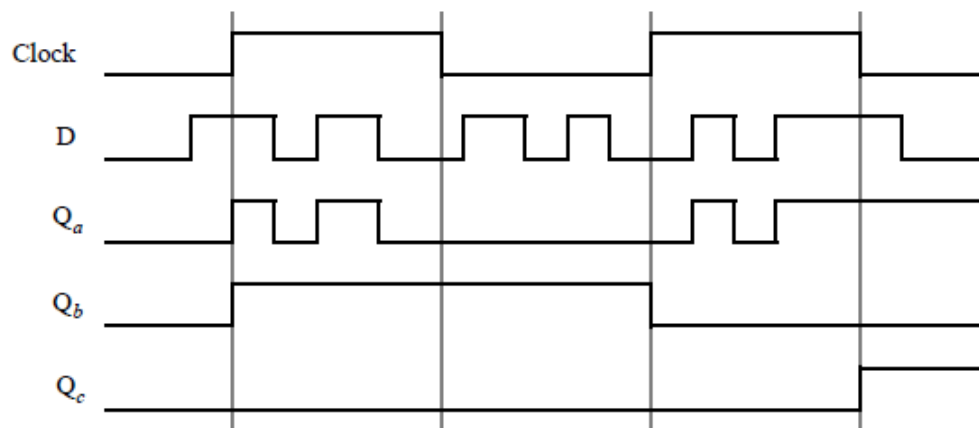


Rys. 5. Schemat przerzutnika D **master-slave**

4. Utwórz projekt latch_FDDP_FDDN, który implementuje układ z rysunku. 6, a. Sprawdź funkcjonowanie dla diagramu czasowego z rys. 6, b.



(a) Circuit



(b) Timing diagram

- (a) Schemat
(b) diagram czasowy

Rys. 6. Schemat i diagramy czasowe dla projektu **latch_FDDP_FDDN**

Aby opisać zatrząsk typu D (Rysunek 6), należy użyć przypisania proceduralnego języka Verilog (rysunek 7). Podobnie należy uczynić, aby opisać projekty przerzutników: **FDD_posedge** i **FDD_negedge**.

```
module D_latch (D, Clk, Q);
    input D, Clk;
    output reg Q;
    always @ (D, Clk)
        if (Clk)
            Q = D;
endmodule
```

Rys. 7. Proceduralny styl kodu **Verilog**, opisujący zatrząsk typu **D**

Wdrożyć projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
D	SW[0]
clk	SW[1]
Qa	LEDR[0]
Qb	LEDR[1]
Qc	LEDR[2]

5. Utwórz projekt **adder_A_B_8_bits**, który dodaje dwie 8-bitowe liczby binarne A i B, wynik jest wyświetlany jako liczba szesnastkowa na ekranach HEX5 i HEX4. Liczby A i B są również wyświetlane w zapisie heksadecymalnym na wyświetlaczach 7-segmentowych: A na HEX3 i HEX2, B na HEX1 i HEX0. Wartości liczb A i B wprowadza się za pomocą przełączników SW7-0.

Wskazówka: Użyj multipleksowania podczas wprowadzania liczb A i B, najpierw wprowadź wartość liczby A i zapamiętaj ją w rejestrze, a następnie wprowadź wartość liczby B. Aby zapamiętać numer A, użyj rejestru **reg_N_bits_with_aret** dla N bitów z asynchronicznym resetem. Zaimplementować projekt na płycie z następującymi przypisaniami pinów:

Wyprowadzenia	Piny
A, B	SW[7:0]
A	HEX3, HEX2
B	HEX1, HEX0
S = A + B	HEX5, HEX4
cout	LEDR[0]
aret	KEY0
clk	KEY1